



Trends Bird Limited

Assignment for Backend Developer Intern

Item	Details
Role	Backend Developer Intern
Assignment	Implement the Ecommerce Admin dashboard from the provided Module below
Submission Deadline	1 August 2026, 11:59 PM
Submission Form	Submission Link

Please read this full document before you begin.

Every requirement here forms part of the evaluation. The backend is what we grade. The frontend exists so that the backend can be seen working. Ui design quality carries less marks; correctness, access control and completeness carry all of them.

1. Overview

You will build the **admin dashboard of an e-commerce platform**

The assignment has two parts. A **REST API** covering the nine modules in Section 3, and a **dashboard frontend** that exercises every one of them. There is no storefront and no customer-facing side.

We are assessing:

- Whether your role and permission system actually enforces access, on the API and not only in the interface
- Whether you can model a real catalog domain: nested categories, attributes, variants and shared media
- How you structure a Node.js codebase, validate input and handle errors
- Whether you can follow a written specification end to end

2. Technology Requirements

Area	Requirement
Database	PostgreSQL (Mandatory)
Runtime	Node.js. Mandatory. Any current LTS version; state it in your README.
Framework	NestJS is preferred and is what we use. Express or Fastify are fully acceptable.
Language	TypeScript is preferred. JavaScript is accepted and is not penalised.
Data access	Your choice: Prisma (preferred), TypeORM, Drizzle, Knex, Sequelize or plain SQL. Migrations must be committed and runnable.
Authentication	JWT access token plus refresh token. Mandatory. Session-only or single-token auth does not meet the requirement.
Passwords	bcrypt or argon2. Never stored or returned in a readable form.

Area	Requirement
Validation	Any library : class-validator, zod, joi. Having one is mandatory.
Frontend	React or Next.js preferred , tailwindcss; any framework accepted. Component libraries such as MUI, Ant Design, shadcn/ui or Bootstrap are encouraged

3. Modules

There are nine modules:

Build them in that order. Each depends on the ones before it, and Product depends on almost all of them.

#	Module	What it covers	Permission prefix
1	Authentication	Login, access and refresh tokens, current session, logout.	—
2	Permission	Defines every capability in the system, grouped by module.	permission
3	Role	Bundles permissions into a named job function.	role
4	User	Dashboard accounts, each holding one role.	user
5	Media	Shared library of uploaded files that other modules attach from.	media
6	Category	Nested category tree a product is filed under.	category
7	Brand	The manufacturer or label a product belongs to.	brand
8	Attribute	Dimensions a product varies along, and their values.	attribute
9	Product	Products, variants, and their categories, brand and media.	product

Section 4 sets the access-control rules that apply to all nine. Section 5 covers them one by one.

4. Access Control

This applies to every module and is the heaviest single item in the marking scheme.

4.1 How access is decided

User holds one Role. A Role holds many Permissions. A route declares the permission it needs. The request succeeds only if the user's role holds that permission.

Permissions are named `module:action` in lower case: `product:create`, `media:upload`, `role:update`.

4.2 What the API must do

- **Authenticate every request by default.** Register the auth guard globally so a new route is protected without anyone remembering to protect it. Public routes `login`, `refresh`, `logout` opt out explicitly.
- **Check permissions with a second guard.** It reads the permission a route declares and compares it against the permissions the user's role holds.
- Return `401` when the token is missing, malformed, expired, wrongly signed, or the account is inactive.
- Return `403` when the token is valid but the role lacks the required permission.
- Return `400` or `422` for validation failures, `404` when a record is missing, and `409` for conflicts such as a duplicate slug. A predictable bad input must never produce a `500`.

4.3 Permissions to implement

Module	Permissions
Dashboard	<code>dashboard:watch</code>
Permission	<code>permission:watch</code> , <code>permission:create</code> , <code>permission:read</code> , <code>permission:update</code> , <code>permission:delete</code>
Role	<code>role:watch</code> , <code>role:create</code> , <code>role:read</code> , <code>role:update</code> , <code>role:delete</code>

Module	Permissions
User	user:watch, user:create, user:read, user:update, user:delete
Media	media:watch, media:read, media:upload, media:write, media:delete
Category	category:watch, category:create, category:read, category:update, category:delete
Brand	brand:watch, brand:create, brand:read, brand:update, brand:delete
Attribute	attribute:watch, attribute:create, attribute:read, attribute:update, attribute:delete
Product	product:watch, product:create, product:read, product:update, product:delete

watch means *may see this menu entry and open this screen*. read means *may read the records*. Drive your sidebar from watch.

4.4 Seeding

Your repository must contain a seed script that, on an empty database, creates every permission above, a super administrator role holding all of them, and one super administrator user. Put the credentials in your README.

Also seed a **second, deliberately limited user**, catalog access only, no permission, role or user access. It lets us verify your 403 behaviour in seconds.

Implementation of proper access control for the frontend also.

A permission-aware interface is required, but it is convenience only. We will take a token belonging to a low-privilege user and call your endpoints directly with Postman.

If a delete succeeds for a user without the delete permission, that module scores zero regardless of how the interface behaves.

The routes most often left unguarded: file uploads, because the request is multipart; nested routes such as attribute values, product variants and role permission assignment; and the routes that create permissions, roles and users. Check every one of these yourself before submitting.

5. Module Requirements

Each module below lists the fields it holds and the behaviour it must support. **Field lists describe what is stored, not how to model it**: column types, keys, join tables and naming are your decisions. Add fields your design needs; do not drop the ones listed.

API routes are not specified. Design your own and document them.

5.1 Authentication

Sign in, stay signed in without a long-lived credential in the browser, and sign out for real.

Fields. email, password (hashed), refresh token (stored so it can be revoked), active flag.

Must support

- Login with email and password, issuing a **short-lived access token** (10 to 15 minutes) and a **long-lived refresh token** (7 to 30 days).
- The same error response for a wrong email and a wrong password. Never reveal which was wrong.
- A session endpoint returning the current user, their role, and a **flat list of their permission names**. The frontend needs this list to decide what to render.
- Refresh: exchange a valid refresh token for a new access token, **rotating the refresh token** at the same time.
- Logout: revoke the refresh token server side. The old token must stop working. Clearing a browser cookie is not logout.
- An inactive user can neither log in nor refresh, even with an unexpired token.

- No public sign-up. Accounts are created through the User module.
- Choose **HttpOnly cookies** or an **Authorization: Bearer header** and document the choice. Cookies also require CORS credentials and CSRF handling.
- No response anywhere returns a password hash or a refresh token.

5.2 Permission

Defines the vocabulary of the system. Build this first as everything else depends on it.

Permissions live at two levels: a **group** per module (Product), and one **permission** per action inside it (product:create). This is what lets the interface show a module-by-action grid instead of a flat list of a hundred checkboxes.

Fields. **Group** — name (the module name), description. **Permission** — name (product:create, unique across the system), description, parent group.

Must support

- Create a group and select its actions in one step. Naming a group Product and ticking Create and Read produces product:create and product:read.
- The standard action set: create, read, update, delete, plus watch, upload, write, approve and status where a module needs them.
- A **custom action name**, for a capability outside the standard set.
- A list grouped by module, with search and pagination.
- Editing a group, including adding and removing actions.
- Permission names are unique, and are normalised or rejected if they arrive in mixed case or with spaces.
- Deleting a permission must not leave roles pointing at something that no longer exists. Cascade the role links or refuse the delete, and say which you chose.

About the screenshots in this document.

The images below show what these features contain a list, a form, a permission grid. They are included so the requirements are easy to picture.

Do not reproduce this visual design.

Access Control

Access Control > Permission

Search...

dd/mm/yyyy - dd/mm/yyyy

10

+ New

☐	Module Name	Create	Read	Update	Delete	Approve	Status	Upload	Watch	Write	Action
☐	Attribute	☒	☒	☒	☒	☐	☐	☐	☒	☐	
☐	Brand	☒	☒	☒	☒	☐	☐	☐	☒	☐	
☐	Cart	☒	☒	☒	☒	☐	☐	☐	☐	☐	
☐	Cart Discount	☒	☒	☒	☒	☐	☐	☐	☒	☐	
☐	Catalog	☐	☐	☐	☐	☐	☐	☐	☒	☐	
☐	Catalog Discount	☒	☒	☒	☒	☐	☐	☐	☒	☐	
☐	Category	☒	☒	☒	☒	☐	☐	☐	☒	☐	
☐	Coupon Discount	☒	☒	☒	☒	☐	☐	☐	☒	☐	
☐	Customer	☒	☒	☒	☒	☐	☐	☐	☒	☐	
☐	Dashboard	☐	☐	☐	☐	☐	☐	☐	☒	☐	

< Previous

123

Next >

Go To

1

of 3

Go

Permission list: module rows against action columns. Reference only, do not copy the design.

Permission

Permission > New

← Back

Permission Group Information

Group Name (Module Name)

Enter group name

Description

Enter description

Permissions

☐ Create

☐ Read

☐ Update

☐ Delete

☐ Approve

☐ Status

☐ Upload

☐ Watch

☐ Write

☐ Custom permission

Enter permission name

Cancel

Reset

Create

Creating a permission group: name, description, action checkboxes, custom permission. Reference only, do not copy the design.

5.3 Role

A named bundle of permissions that can be handed to a user.

Fields. name (unique), description, status, permissions.

Must support

- Creating a role and granting it a set of permissions in one step.
- Reading a role with its full permission set, so the edit screen can pre-tick the grid.
- Adding and removing individual permissions. A revoked permission must take effect for users holding that role.
- A grant-all shortcut, for building an administrator role.
- A list with search and pagination, ideally showing how many users hold each role.
- Refusing to delete a role while users still hold it.

- Refusing a change that would leave nobody able to manage roles , for example stripping `role:update` from the only role that has it. A small guard, but it tells us you thought about consequences.

Role

Role

Search...

Search

Filter

View

Download

Share

Refresh

Print

Export

+ New

ID	Name	Status	Action
22	Super Admin	Active	
22	Admin	Active	
22	Planning	Active	
22	MCD	Active	

Role list. Reference only, do not copy the design.

Search

Role

Role > New

Role Information

Role Name

Enter role name

Status

Select role status

☐	Module	☐ Create	☐ Read	☐ Update	☐ Delete
☐	Dashboard	☐	☐	☐	☐
☐	CRM	☐	☐	☐	☐
☐	Product	☐	☐	☐	☐
☐	Warehouse - MCD	☐	☐	☐	☐
☐	Production	☐	☐	☐	☐
☐	Inventory - FG	☐	☐	☐	☐
☐	Planning	☐	☐	☐	☐
☐	Requisition	☐	☐	☐	☐
☐	Challans	☐	☐	☐	☐
☐	Disputes	☐	☐	☐	☐

Cancel

Reset

Draft

Preview

Create

© 2026 Giant BD Co. All Rights Reserved.

Creating a role: name, status, and the module-by-action permission grid. Reference only, do not copy the design.

5.4 User

The people who can sign in. Each holds exactly one role.

Fields. name, email (unique), password (hashed), phone, gender, avatar, role(required), active flag.

Must support

- Creating a user with an explicit role. The role is required, never defaulted.
- A list showing role and status, with search by name or email, filters for role and status, and pagination.
- Updating a user, including **changing their role**. State in your README whether that takes effect on the next request or the next refresh.
- Activating and deactivating. A deactivated user can neither log in nor refresh.
- Deleting a user. State whether it is a hard or a soft delete.
- Preventing self-escalation: a user cannot change their own role or permissions.

Search

User

User

Search...

+ New

☐	ID	Name	Role	Gender	Phone	Email	Status	Action
☐	22	John Doe	Raw Material	Male	+880171111111	rm@gaintbd.com	Active	
☐	22	John Doe	Compound	Male	+880171111111	com@gaintbd.com	Active	
☐	22	John Doe	Planning	Male	+880171111111	pln@gaintbd.com	Active	
☐	22	John Doe	Inventory	Male	+880171111111	invty@gaintbd.com	Active	
☐	22	John Doe	Shipment	Male	+880171111111	ship@gaintbd.com	Active	

← Previous

1 2 3 5 6 7 8 9 10 ... 499

Next →

Go To

User list showing the assigned role and status. Reference only, do not copy the design.

5.5 Media

A shared library of uploaded files. Products, categories and brands attach from it rather than each owning its own uploads. Build it before the Product module.

Field	Holds
file name, stored path, public URL	The original name, where the file lives, and the URL the frontend displays.
mime type, type, size	For example image/jpeg; image or video, so the library can be filtered; size in bytes.
width, height	Image dimensions.
thumbnail	A generated small version, used in the library grid and in list screens.
alt text, title	Editable after upload.
uploaded by	Which user uploaded it.

Must support

- Uploading one file, and several files in one request.
- **Validating every upload:** mime types against an allow-list, and a maximum size. Do not trust the file extension or the client-supplied content type.
- Generating a **thumbnail** for every image.
- A library list with pagination, search and a filter by type.
- Editing metadata, at minimum alt text and title.
- Deleting an asset: remove the stored file as well as the record. Decide what happens when it is still attached to a product — refuse, or detach cleanly. Leaving a product pointing at a missing file is not acceptable.
- **One asset attaching to many products.** Uploading the same image again for a second product is a design failure.

5.6 Category

The nested tree a product is filed under, so that `Electronics > Phones > Android` is one path through one table.

Fields. name, slug (unique), description, image, parent category, active flag, sort order.

Must support

- Full create, read, update and delete, with unlimited nesting depth.
- Returning the category **tree**, not only a flat list, so the interface can render it and offer a parent picker.
- A unique slug, enforced by the database rather than by application code alone.
- **Rejecting cycles.** A category must never become its own ancestor. We will test this.
- Not orphaning children or products on delete: refuse, or reassign, and document the choice.
- A product belonging to **many** categories, and a category holding many products.

5.7 Brand

The manufacturer or label a product belongs to. The simplest module here, so there is no excuse for it to be incomplete.

Fields. name (unique), slug (unique), logo, status, description.

Must support

- Full create, read, update and delete, with search, pagination and a status filter.
- At most one brand per product; many products per brand.
- Refusing deletion while products still reference the brand.

5.8 Attribute

The dimensions a product varies along — Size, Colour, Storage — and the values each can take. This module decides how hard the Product module will be.

Fields. Attribute — name (unique), slug, type (dropdown, radio, checkbox, colour swatch, image swatch), values.

Value — value, slug, reference value (a hex code for a colour attribute, a media reference for an image attribute), parent attribute.

Must support

- Full create, read, update and delete for attributes.
- Adding, editing and removing values without recreating the attribute.
- A value being unique within its attribute. Red may exist once under Colour, and separately under another attribute.
- Refusing to delete an attribute or a value already used by a product variant. Deleting it silently corrupts every variant built on it.

5.9 Product

The centre of the assignment. A product pulls everything together: it sits in categories, belongs to a brand, shows media from the library, and either carries its own price and stock or splits into variants.

The distinction that matters

- A **simple product** has no variants. Price, sale price and stock live on the product. A book, a poster.
- A **variable product** has variants. Price, sale price and stock live on **each variant**, because a Red / XL shirt sells out independently of a Blue / S shirt.

A flag records which kind it is, and your validation must follow it: reject a price on a variable product, and a variant on a simple one.

Product fields	Holds
name, slug (unique), SKU (unique)	Identity. On a variable product the meaningful SKU sits on each variant.
short description, long description	Summary text and full description. Sanitise it if you accept HTML.

Product fields	Holds
has variants	The simple-or-variable flag.
price, sale price, stock, stock status	Used only when the product has no variants. Sale price must not exceed price. Derive stock status from the count.
weight, active flag, featured flag, sort order	Shipping weight, whether published, whether promoted, manual position.
brand, categories	At most one brand. Many categories.
media, variants	Attached media, and the variants when the product is variable.

Variant fields	Holds
SKU	Unique across all variants.
attribute values	The combination defining this variant, for example Colour = Red and Size = XL.
price, sale price, stock, stock status	This variant's own figures.
low stock threshold, weight, active flag	When it counts as low stock, shipping weight, whether purchasable.
media	Images specific to this variant.

Media attachment fields	Holds
media reference	Which library asset is attached.

Media attachment fields	Holds
attached to	The product, a specific variant, or a specific attribute value. Attaching to an attribute value is how picking Red shows the right photos without duplicating them on every Red variant.
is thumbnail	Exactly one per product , and at most one per variant.
is gallery, sort order	Whether it appears in the carousel, and its position.

Must support

- Creating a simple product with its own price, sale price and stock.
- Creating a variable product: choose the participating attributes and values, then produce the variant combinations. Generating the full set and letting the administrator remove combinations they do not stock is friendliest; defining variants one at a time is acceptable.
- Giving each variant its own SKU, price, sale price and stock.
- Attaching and detaching **many categories**, and setting the **brand**.
- Attaching media from the library, **setting exactly one thumbnail**, marking gallery membership and **reordering the gallery**.
- Attaching media to a **specific variant** and to a **specific attribute value**.
- A product list with pagination, search by name or SKU, filters for category, brand and status, and sorting. Each row needs the thumbnail, brand, categories and price or price range.
- Reading one product with everything an edit screen needs: categories, brand, media, variants and each variant's attribute values.
- Updating a product: adding and removing variants, swapping the thumbnail, reordering the gallery, changing categories.
- Deleting a product: variants and media **attachments** go with it, but the media **assets survive**, because other products may use them.

Validation we will test

- Duplicate slug, duplicate product SKU or duplicate variant SKU : rejected.
- Sale price above price, negative price, negative stock : rejected.
- Two variants of one product with the identical attribute combination : rejected.
- A variant referencing an attribute value that does not exist : rejected.
- A second thumbnail on one product : rejected, or it demotes the previous one. Never two at once.
- Creating a product together with its categories, media and variants must be **atomic**. If the variants fail validation, no half-built product may survive. Use a transaction.

6. Frontend Requirements

The frontend proves the backend works. It is required, and deliberately the lightest part of the marking scheme.

6.1 Screens

Screen	Must let a user
Login	Sign in, see a clear error on bad credentials, and land in the dashboard.
Dashboard shell	Navigate via a sidebar, see the signed-in user and role, and log out. Entries the user has no permission for are not shown.
Permission	See permissions grouped by module, and create a group by naming it and ticking its actions.
Role	Create and edit a role, granting and revoking permissions through a module-by-action grid with select-all. Every reviewer will use this screen, so it must be usable.
User	Create and edit a user, pick a role, toggle active status, and see role and status in the list.
Media library	Upload files, browse thumbnails, edit alt text and title, and delete. Show upload progress and errors.

Screen	Must let a user
Category	See the tree, create and edit a category, pick a parent and an image, set status and order.
Brand	Create and edit a brand with a logo picked from the media library.
Attribute	Create and edit an attribute, choose its type, and manage its values, including a colour value.
Product list	Browse a paginated table with thumbnail, name, brand, categories, price, stock and status, with search and filters served by the API.
Product form	The most important screen. Sectioned or tabbed: details; brand and categories; media, with a library picker, thumbnail selection and gallery reordering; and variants, where attributes and values are chosen, combinations produced, and each variant given its own SKU, price, stock and images.

6.2 Behaviour

- The session survives a page refresh, restored from the session endpoint on load.
- When a request fails because the access token expired, **refresh once and retry** transparently. If the refresh also fails, clear the session and return to login. Guard against parallel refreshes with a single in-flight promise.
- Logout calls the backend so the refresh token is actually revoked.
- Actions the user lacks permission for are hidden or disabled, driven by the permission list from the session endpoint.
- A 403 shows a clear message, not a blank screen or a raw error object.
- Every list and form has loading, empty and error states, and shows API validation errors against the right fields.
- Pagination, search and filtering call the API. Do not fetch everything and filter in the browser.

6.3 Design

- A standard dashboard layout sidebar, top bar, content area : built with a component library.

- Do not reproduce the design of the screenshots in this document.
- Polish, animation and dark mode earn **no marks**. Completeness of the flows earns marks.

7. Code Quality

Your source code is reviewed and forms a significant part of the evaluation.

- One module per feature, with routing, business logic and data access separated. We should be able to guess which file holds a given behaviour without searching.
- Every body, query parameter and route parameter validated before it reaches your business logic.
- One consistent error shape and one consistent success shape, produced centrally. No stack traces, raw database errors or internal paths in a response.
- Multi-table writes wrapped in transactions.
- No secret, token or database URL committed. Provide a `.env.example` listing every variable.
- Commit incrementally with clear messages. A repository containing a single commit with the entire project will be marked down. We are interested in how you work, not only in the final result.

8. Partial Submissions

Partial submissions are accepted and encouraged. We would far rather review five modules built carefully, with access control working properly, than nine built badly. Modules you did not build score zero, but a broken module costs you more than a missing one.

Follow the build order in Section 3, and state clearly in your README which modules are complete, which are partial, and which were not attempted. Presenting incomplete work as finished is treated more seriously than leaving it out.

9. Submission Requirements

All of the following are mandatory. A submission missing any one is treated as incomplete.

1. **GitHub repository** : public, with the full source code, commit history and a README. If it must stay private, add our reviewer account and note it on the submission form.
2. **Migrations and a seed script** :with the exact commands to run them, so we can go from an empty database to a working one.
3. **An API collection** : Postman, Insomnia, Thunder Client or a Swagger endpoint, covering the routes you built.
4. **Live Link including user credentials**

Submissions are accepted only through the form linked at the top of this document; links sent by email cannot be tracked.

9.1 README Requirements

- Setup steps, environment variables, and the migrate, seed and run commands
- The seeded account credentials
- Your token strategy , cookies or header and any notable design decisions
- A module-by-module status list: complete, partial, or not attempted
- Known issues

10. Evaluation Criteria

Criteria	Weight
Access control: guards, permission enforcement, correct 401 and 403, no unguarded route	25%
Product module: variants, media, thumbnail, categories, brand, validation	20%
Authentication: token handling, refresh with rotation, real logout	15%
Permission, Role and User modules	15%
Category, Brand, Attribute and Media modules	15%
Code quality, structure, validation, error handling, version control practice	7%
Ui and README quality	3%

Visual design carries no weight. Considered as a bonus, after the criteria above: TypeScript, refresh-token reuse detection, rate limiting on login, an audit log, tests, and Docker Compose.

11. Timeline

Milestone	Date
Assignment released	26 July 2026
Questions and clarifications	Any time before the deadline; early contact is encouraged
Submission deadline	1 August 2026, 11:59 PM
Shortlist and code walkthrough	Within approximately one week of the deadline

12. Assignment Rules

- Read this document in full before beginning.
- PostgreSQL and Node.js are fixed. Every other technology choice is yours.
- Submit original work. Copying another candidate's repository, or submitting a purchased or downloaded template, will result in immediate disqualification.
- AI coding assistants are permitted, but you must fully understand the code you submit. Shortlisted candidates walk through their implementation and answer questions on it. Code you cannot explain counts against you more than code you did not write.
- If something here is genuinely ambiguous, contact us. If you do not receive a response in time, make a reasonable decision and record the assumption in your README.
- The deadline is firm. Late submissions will not be evaluated; submit what you have completed rather than nothing at all.

13. Frequently Asked Questions

Is NestJS mandatory?

No. Express or Fastify are fully acceptable. Only Node.js and PostgreSQL are fixed.

Do I have to use TypeScript?

No. JavaScript is accepted and is not penalised. TypeScript is preferred.

Cookies or the Authorization header?

Either. Pick one, implement it properly, and document it in your README.

Do I need to build a storefront or a checkout?

No. Admin dashboard only. No orders, no cart, no customers.

Can I use a UI component library on the frontend?

Yes, and you should. Design is not graded, so do not spend time on custom CSS.

How complete does the frontend have to be?

Complete enough that every backend feature can be demonstrated through it. Nothing more.

Can I skip a module?

Yes. Partial submissions are accepted. Follow the build order and record what you completed in your README.

Do I need to write tests?

No, tests are optional. A few around the permission guard are valued, but never at the cost of working features.

Should I seed sample products and categories?

It is helpful , but only the permissions, roles and users seed is required.